费曼口述：AI大模型 好的Prompt具备的核心要素

说话人1: 费曼口述，AI大模型优质Prompt的核心要素与设计方法，共十七道关键问题，由浅入深分四个阶段，覆盖基础认知、要素拆解、进阶技巧、工程落地与体系化复用，讲清每个要素的职责边界、设计逻辑与Java后端场景的落地方法。

第一阶段：基础认知与核心定位

说话人1: 问题一，什么是Prompt？它在大语言模型接收请求、理解任务和生成结果的过程中承担什么作用？
说话人1: 答，Prompt也就是提示词，是我们传给大模型的全部输入信息，包含任务、材料、规则、格式所有内容。
说话人1: 它的核心作用，可以用一句话概括：大模型是基于概率生成内容的，它没有读心术，所有输出都只能基于你给的Prompt，除此之外它不知道任何额外信息。打个比方，Prompt就像是一张完整的考卷，既给题目、给材料，也给答题规则、给评分标准；模型就是考生，只能照着卷子上的内容答题，卷子上没写的，它要么靠训练知识瞎猜，要么就答非所问。
说话人1: 很多人觉得模型不够聪明，其实大部分时候不是模型不行，是Prompt没写清楚，相当于考卷出得模糊，考生自然答不对。

说话人1: 问题二，一个好的 Prompt 应该具备哪些核心要素？角色设定、任务目标、背景上下文、输入材料、执行步骤、约束条件、输出格式和质量标准分别解决什么问题？
说话人1: 答，一个完整可落地的优质Prompt，有八个核心要素，各司其职，共同把模糊需求变成可执行、可验收的指令。
说话人1: 角色设定，解决“用什么身份视角回答”的问题，决定了回答的专业度、语气和立场；任务目标，解决“到底要做什么”的问题，是整个Prompt的核心，所有内容都围绕它展开；背景上下文，解决“在什么场景下做”的问题，给模型补齐业务、技术、环境信息，避免给通用答案；输入材料，解决“基于什么内容做”的问题，是模型加工的原始数据，比如代码片段、文档原文；执行步骤，解决“按什么顺序做”的问题，拆解复杂任务，避免模型跳步漏项；约束条件，解决“不能做什么”的问题，划清边界，防止模型自由发挥出界；输出格式，解决“交付成什么样”的问题，规定输出的结构、格式，方便阅读和程序解析；质量标准，解决“做到什么程度算合格”的问题，把模糊的“质量好”变成可逐条核对的验收线。
说话人1: 八个要素从身份、目标、背景、原料、流程、边界、交付、验收八个维度锁死任务，模型自由发挥的空间越小，结果越贴合预期。

说话人1: 问题三，为什么 Prompt 必须明确任务目标？如何区分模糊的需求描述与可执行、可验收的任务指令？
说话人1: 答，因为大模型不会主动猜测你的真实意图，目标越模糊，模型的自由发挥空间越大，结果跑偏的概率就越高。你不说清楚要什么，它就只能按最通用、最保险的方式回答，往往解决不了你的具体问题。
说话人1: 模糊描述和可执行指令，核心差三个东西：有没有明确的动作对象、有没有明确的产出物、有没有可验证的验收标准。比如“优化一下这段Java代码”就是模糊描述，优化什么方向？性能还是可读性？改到什么程度算好？都没说，模型只能泛泛改一下。
说话人1: 可验收的指令应该是：“基于JDK21优化这段用户查询接口的Java代码，目标是将平均响应耗时从500ms降到200ms以内，不得修改原有业务逻辑，不得新增第三方依赖，最终输出优化后的完整代码加性能说明。” 动作、对象、目标、约束、产出全都说清楚，模型才能精准执行。
说话人1: 判断标准很简单：看完指令，你能不能立刻知道做完之后是什么样子、怎么检验合格。能，就是可执行的；不能，就是模糊的。

第二阶段：核心要素逐项拆解

说话人1: 问题四，角色设定有什么作用？什么时候需要指定专家身份、专业领域或工作职责？为什么仅仅赋予模型一个专家角色并不能保证答案的准确性？
说话人1: 答，角色设定的核心作用，是切换模型的回答视角和知识权重，让它用对应领域的思维方式、术语体系、经验常识来回答问题。比如你设定成“12年Java架构师”，它回答就会偏向工程实践、性能优化、生产踩坑，不会给你讲入门科普；设定成“初级讲师”，它就会讲得浅显易懂，适合新手。
说话人1: 什么时候需要加角色？专业领域问题、需要特定身份立场、需要统一输出风格的时候加，比如技术方案评审、代码审查、文档撰写。简单常识问题、纯信息查询就没必要加。
说话人1: 为什么光有角色保证不了准确性？因为角色只是调整了回答的语气和视角，改变不了模型的事实知识。你给它安个架构师的头衔，它也不会凭空知道你们公司的代码规范、业务逻辑、技术栈版本，该幻觉还是会幻觉，该说错还是会说错。角色是加分项，不是事实保证，准确的前提还是要有准确的输入材料和约束。

说话人1: 问题五，背景上下文为什么重要？如何提供必要的业务背景、技术版本、目标人群和现有条件，同时避免无关信息干扰？
说话人1: 答，因为模型训练的都是通用知识，它不知道你的具体场景。不给背景，它就只能给通用答案，看着都对，用起来全不适用。比如你不说用的是SpringBoot3.5，模型可能按SpringBoot2给方案，用到javax包，放到你的项目里直接编译报错。
说话人1: 提供背景的原则，是“只给和任务直接相关的，无关的一律去掉”。技术任务就说清楚JDK版本、框架版本、当前环境、现存问题；业务任务就说清楚目标人群、业务目标、现有条件。
说话人1: 为什么要避免无关信息？第一，无关信息会稀释有效指令，模型容易抓不住重点；第二，多余的信息可能误导模型，让它往错误方向发挥；第三，浪费Token，增加成本。所以背景不是越多越好，而是越精准越好。

说话人1: 问题六，输入材料与任务指令有什么区别？如何通过明确的数据范围、资料来源、处理对象和禁止事项，减少模型误解或擅自补充信息？
说话人1: 答，两者完全不是一回事：任务指令是告诉模型“你要做什么”，是动作要求；输入材料是告诉模型“你基于什么东西做”，是加工原料。比如“审查这段代码的性能问题”是指令，你贴的那一段Java代码就是输入材料。
说话人1: 减少误解和擅自补充，核心是“划清原料边界，明确禁止规则”。第一，明确标注哪些是原始材料，哪些是要求，不要混在一起；第二，明确数据范围和资料来源，比如“仅基于下面提供的接口文档回答，不要使用文档外的API”；第三，明确禁止事项，比如“禁止编造不存在的类和方法，代码必须能在JDK21下直接编译”。
说话人1: 本质就是把模型框死在你给的材料里，不让它自己从训练知识里瞎补东西，从根源减少幻觉和跑偏。

说话人1: 问题七，约束条件应该包含哪些内容？如何约束技术版本、实现范围、资源预算、安全要求、禁止行为和事实依据？
说话人1: 答，约束条件就是给模型画红线，告诉它什么能做、什么绝对不能做，一共六类常见约束。
说话人1: 第一，技术版本约束，比如必须使用JDK21、SpringBoot3.5.x，禁止使用已废弃的API；第二，实现范围约束，比如只能修改Service层代码，不能改动数据库表结构，不能新增外部依赖；第三，资源预算约束，比如不能引入新的中间件，不能超过现有服务器资源；第四，安全要求，比如必须做参数校验、敏感数据必须脱敏、禁止SQL拼接；第五，禁止行为，比如不能硬编码配置、不能写空catch吞异常、不能编造不存在的技术点；第六，事实依据约束，比如所有结论必须基于提供的材料，无来源的内容必须标注为推断。
说话人1: 约束越具体，模型输出的结果越能直接落地，不用你再返工修改。很多时候代码拿过来不能用，就是因为没加版本、环境约束，模型按自己默认的版本写的，和你项目不兼容。

说话人1: 问题八，为什么需要明确输出格式？如何通过 Markdown 标题、表格、JSON Schema、代码结构和固定模板，提高结果的可读性与可用性？
说话人1: 答，因为模型默认输出是松散的自然语言，人读着费劲，程序也没法直接用。明确输出格式，一是提升可读性，结构清晰不用自己整理；二是提升可用性，结构化的结果可以直接被程序解析，不用人工转换。
说话人1: 具体怎么设计格式，看使用场景：给人看的内容，用Markdown分级标题、列表、表格，逻辑清晰重点突出；需要程序解析的内容，给明确的JSON Schema，规定字段名、字段类型、取值范围，模

型输出可以直接反序列化成Java对象；代码类任务，指定编程语言、包结构、注释要求、是否包含单元测试；有固定模板的任务，直接给填空模板，模型往里面填内容就行。
说话人1: 比如你要求模型返回接口的错误码列表，指定用表格输出，字段包含错误码、含义、触发场景、解决方案，拿到手直接就能放进文档里，不用自己重新排版。

说话人1: 问题九，什么是输出质量标准？如何把“内容专业、质量高、足够详细”等模糊要求，转换为可以逐项检查的验收条件？
说话人1: 答，质量标准就是任务的及格线，是一条条可以对照检查的硬规则，满足就算合格，不满足就算不合格，不靠主观感觉判断。
说话人1: 转换的核心方法，就是把形容词替换成可验证的硬指标。比如“内容专业”，换成“所有技术点符合JDK21官方规范，每个优化点附带底层原理说明”；“质量高”，换成“代码无语法错误、包含参数校验、异常处理、单元测试，可直接编译运行”；“足够详细”，换成“每个步骤包含操作方法、风险点、回滚方案三项内容，缺一不可”。
说话人1: 判断标准好不好，就看你能不能照着一条条打勾。能打勾，就是好的验收标准；还需要主观判断好不好，就是不合格的标准。

说话人1: 问题十，示例与少样本提示有什么作用？什么情况下应该提供正例、反例或参考模板？示例过多或存在错误时可能产生什么影响？
说话人1: 答，示例的作用，是用实际样例代替抽象描述，让模型更精准地理解你的要求。很多时候你说半天格式要求，不如直接给一个正确样例，模型一看就懂。
说话人1: 什么时候用正例？需要明确输出格式、回答风格、逻辑结构的时候，给1到2个正确样例，模型照着对齐就行。什么时候用反例？需要明确禁止什么、容易踩坑的地方，给错误示例加错误原因，避免模型踩雷。
说话人1: 示例过多的坏处：第一，占用大量上下文，稀释核心指令，反而让模型抓不住重点；第二，过度限制模型，让它只会机械模仿，失去灵活性。示例错误的坏处更严重，模型会跟着错误示例学，输出全是错的，比不给示例效果还差。
说话人1: 所以示例贵精不贵多，1到3个足够，而且必须准确无误，不然适得其反。

第三阶段：进阶技巧与优化方法

说话人1: 问题十一，如何在 Prompt 中要求模型处理不确定信息、缺失条件和相互矛盾的材料？怎样减少幻觉、无依据推断和虚构引用？
说话人1: 答，核心思路是：提前给规则，让模型遇到不确定的情况主动说出来，而不是硬着头皮瞎编。
说话人1: 具体可以加三条规则：第一，信息不足时，明确说明缺少哪些条件，给出假设前提，不要默认编造；第二，材料相互矛盾时，明确指出冲突点，分别说明不同材料的结论，不要自行取舍；第三，所有事实性结论必须标注信息来源，没有来源的推断必须明确说明是“推测”，不能当成事实输出。
说话人1: 减少幻觉最有效的方法，就是两条：一是给足权威输入材料，让模型基于材料回答，不是靠训练记忆；二是明确禁止编造，加溯源要求。尤其是RAG知识库场景，必须加上“仅基于提供的知识库内容回答，知识库中没有的信息请直接告知无法解答，不要自行补充”，能大幅降低幻觉概率。

说话人1: 问题十二，复杂任务为什么需要拆解为多个步骤？如何设计任务分解、执行顺序、中间结果检查和最终验收机制，避免模型遗漏关键环节？
说话人1: 答，因为模型一步到位处理复杂任务，很容易跳步骤、漏环节、逻辑出错，就像人做复杂项目不做计划，也容易漏东西。拆解成步骤，让模型一步步走，每一步都聚焦一个小目标，准确率会高很多。
说话人1: 怎么设计拆解机制？分三层：第一层，拆大阶段，把复杂任务切成几个大步骤，每个步骤有明确的产出物，比如代码审查任务，第一步找问题，第二步分等级，第三步给方案，第四步写代码；第二层，加中间检查点，每完成一步先自检，没问题再进下一步，比如“完成问题排查后，先对照约束条件检查是否有遗漏，再进入优化方案阶段”；第三层，加最终验收，全部完成后，对照质量标准逐条自查，不满足的自行修正。
说话人1: 本质就是把模型当成一个执行员，你给它写好SOP，它照着流程走，出错概率就会大幅下降。

说话人1: 问题十三，什么是 Prompt 的迭代优化？如何通过测试样例、错误记录、输出对比和评估指标，判断某次修改是否真正改善了结果？
说话人1: 答，Prompt不是一次就能写完美的，就像代码要调优一样，通过测试、反馈、修改，越调越好，这个过程就是Prompt迭代优化。
说话人1: 怎么判断修改有没有用，不能凭感觉，要用数据说话。第一步，准备固定的测试样例集，同一场景准备3到5个典型输入，每次改完Prompt都用同样的用例跑，保证对比公平；第二步，设定量化评估指标，比如格式合规率、事实准确率、幻觉出现率、验收项通过率；第三步，每次修改只改一个点，记录修改内容和对应的指标变化，指标变好就保留，变差就回退。
说话人1: 比如原来的Prompt输出JSON格式错误率有30%，加上JSON Schema约束之后降到5%，那这次修改就是有效的。如果改完错误率反而上升，那这个改动就是坏的，要撤回来。
说话人1: 核心原则：迭代要小步快跑，每次只改一个变量，用固定用例验证，用数据判断好坏，不要凭主观感觉。

说话人1: 问题十四，单轮 Prompt、多轮对话上下文与系统指令之间有什么区别？如何管理长期规则、当前任务、历史信息和动态输入，避免上下文冲突？
说话人1: 答，三者作用范围和优先级完全不同。系统指令是全局规则，整个对话全程生效，比如角色设定、通用行为准则，一般放在最开头，一直有效；多轮上下文是历史对话记录，用来保持对话连贯性，让模型记得之前聊过的内容；单轮Prompt是当前这一轮的具体任务指令，针对性最强，优先级最高。
说话人1: 怎么管理避免冲突？第一，分层存放，不变的通用规则放系统指令，本次任务的具体要求放当前用户输入，历史对话只做背景；第二，优先级明确，当前指令 > 系统指令 > 历史上下文，冲突时以高优先级为准；第三，定期清理历史，无关的、太久远的对话要截断，不要什么都堆在上下文里，既浪费Token，又容易干扰当前任务；第四，规则要统一，不要系统指令说“回答要简洁”，当前任务又要求“详细展开”，模型会混乱。
说话人1: 很多人多轮对话越聊越偏，就是因为历史信息太多，稀释了核心指令，加上规则前后冲突，模型不知道该听哪个。

第四阶段：工程落地与体系化复用

说话人1: 问题十五，面向 Java 后端开发与 Spring AI 项目，如何设计一份完整的生产级 Prompt，使模型能够依据指定版本、现有代码、业务需求、工程规范、测试要求和验收证据完成任务？
说话人1: 答，生产级Prompt必须结构完整、约束明确、可验收，对应Java项目场景，按八层结构设计最稳妥。
说话人1: 第一层，角色设定。比如“12年Java后端架构师，精通JDK21、SpringBoot3.5.x、Spring AI，熟悉工业级代码规范、性能优化与故障排查，输出严格遵循工程落地标准”。
说话人1: 第二层，背景上下文。说明项目情况，比如“当前项目是SpringBoot3.5.0 + JDK21的企业知识库RAG系统，采用Maven多模块架构，代码规范遵循阿里巴巴Java开发手册，现有模块代码见下方输入”。
说话人1: 第三层，输入材料。明确标注原始材料，比如“待优化的RAG检索模块代码如下、当前性能压测数据如下、需求文档如下”。
说话人1: 第四层，任务目标。明确动作和产出，比如“审查这段检索模块代码，找出性能瓶颈与安全隐患，给出可落地的优化方案，目标将平均检索耗时降低40%以上”。
说话人1: 第五层，执行步骤。拆解流程，比如“第一步：分析现有代码逻辑与性能瓶颈；第二步：按风险等级列出问题；第三步：给出优化后的完整代码；第四步：说明优化原理与注意事项；第五步：给出单元测试用例；最后对照验收标准自检”。
说话人1: 第六层，约束条件。划清红线，比如“不得引入新的第三方依赖，不得破坏原有API兼容性，修改必须兼容Spring AI 1.1.x，禁止使用已废弃API”。
说话人1: 第七层，输出格式。规定交付结构，比如“按问题描述、风险等级、优化代码、原理说明、测试用例五部分输出，Markdown格式，代码带完整包名和注释”。
说话人1: 第八层，验收标准。可逐条核对，比如“代码可直接编译通过、每个优化点有明确原理依据、包含不少于3个单元测试用例、无语法与规范问题”。
说话人1: 最后再加一条兜底规则：“不确定的内容请明确标注，不要编造不存在的类和方法。” 整套下来，输出的结果基本可以直接投入项目，不用大幅修改。

说话人1: 问题十六，面对知识学习、技术文档生成、代码审查、项目迭代、信息检索和复杂决策等不同场景，Prompt 的核心要素应该如何调整？哪些要素必须保留，哪些可以根据任务简化？
说话人1: 答，调整的核心原则是：任务越复杂、要求越严格，要素越齐全；任务越简单、越通用，要素可以越精简。
说话人1: 有三个要素是所有场景的骨架，绝对不能省：任务目标、约束条件、输出格式。缺了任何一个，结果都会跑偏。
说话人1: 具体场景调整：知识学习场景，重点加角色、执行步骤、质量标准，侧重讲解逻辑和易懂性，输入材料可以少；技术文档生成场景，重点加输入材料、格式模板、目标受众、质量标准，侧重结构规范；代码审查场景，重点加技术版本约束、输入代码、审查维度、输出格式，侧重要点全面准确；项目迭代场景，重点加现有代码、业务需求、工程规范、验收标准，侧重可落地性；信息检索RAG场景，重点加输入材料、引用规则、禁止编造，侧重事实准确；复杂决策场景，重点加背景、评估维度、输出结构，侧重逻辑全面。
说话人1: 总结就是：骨架三要素必留，其余要素按需加减，不要为了全而全，也不要图省事缺关键项。

说话人1: 问题十七，如何建立一套可复用的 Prompt 模板与评估体系，使任务目标、输入信息、执行规则、输出结构和质量验收能够标准化，并通过实际测试持续改进？
说话人1: 答，做成标准化的模板体系+评估闭环，就像项目里的组件库和测试流程，不用每次从零写Prompt，输出质量也能稳定。
说话人1: 第一步，建设模板库。按业务场景分类，比如代码审查模板、文档生成模板、RAG问答模板、方案评审模板，每个模板做成插槽式，预留固定位置：{{角色设定}}、{{任务目标}}、{{输入材料}}、{{约束条件}}、{{输出格式}}、{{验收标准}}，用的时候填空就行，保证结构统一，不会漏项。
说话人1: 第二步，建设测试用例集。每个场景准备3到5个典型测试输入，覆盖正常情况、边界情况、异常情况，作为模板效果的基准测试集。
说话人1: 第三步，建立量化评估指标。每个场景设定可衡量的指标，比如格式合规率、事实准确率、幻觉率、验收项通过率、平均Token消耗，用来量化模板好坏。
说话人1: 第四步，建立迭代闭环。每次修改模板，都用测试用例集跑一遍，对比指标变化，指标提升就更新模板，指标下降就回滚；定期收集线上bad case，补充进测试用例，持续优化模板。
说话人1: 最终效果就是：任何人拿到模板，都能写出合格的Prompt，输出质量稳定，不会因人而异；同时体系可以持续迭代，越用越好。

说话人1: 全部回答完毕。